A Learned Selector for Classical Disk Scheduling

CSE-307 Operating Systems - Track 2 Term Paper

Abstract. No fixed disk-scheduling policy is optimal for every request pattern. This work implements FCFS, SSTF, SCAN, and C-SCAN and trains a small decision tree to choose a policy from an incoming request window. Experiments use reproducible sequential, random, and bursty workloads, disjoint training, validation, and test seeds, and explicit workload shifts. On 660 held-out 50-request windows, the selector achieved 75.61% tie-aware accuracy and 124,574 tracks of movement, improving on the best fixed policy, SSTF (133,276), while approaching an offline oracle (119,296). Confidence was higher on correct than incorrect predictions (0.877 versus 0.778), although ECE=0.148 shows overconfidence remains. A validation-selected random-forest extension improves accuracy and calibration but not seek regret, showing why both classification and operating-system cost must be reported.

1. Problem and Design

Disk scheduling orders pending track requests. FCFS preserves arrival order but can travel unnecessarily; SSTF greedily chooses the nearest request and can starve distant requests; SCAN sweeps like an elevator; and C-SCAN performs a one-directional sweep with a wrap [1]. Their relative cost depends on locality and request order, motivating an adaptive selector.

The simulator uses tracks 0-199. The head resets to 50 for every independent, non-overlapping 50-request window. Following the course convention, SCAN initially moves toward 0, reaches physical 0, then reverses. C-SCAN also moves toward 0, reaches 0, wraps to 199, and counts that wrap. Unit tests reproduce the course queue [98, 183, 37, 122, 14, 124, 65, 67] from head 53: FCFS=640, SSTF=236, SCAN=236, and C-SCAN=386.

For every window all four algorithms run offline; every minimum-cost policy is preserved as an oracle winner. Exact ties remain in raw data but are excluded from primary fitting. The main classifier is an interpretable decision tree (depth 5, minimum leaf size 12, balanced class weights, seed 42), a standard classification method [2]. Its eight pre-decision features are request variance and range, mean absolute gap, gap coefficient of variation, direction consistency, mean distance from the head, and fractions below and above the head. Confidence is the maximum predicted class probability.

2. Experimental Method

Sequential workloads are mostly monotonic with small positional noise; random workloads are uniform over all tracks; and bursty workloads contain ordered clusters around several hot spots. Twenty windows per workload are generated for every seed. Seeds 1-20 train, 21-30 validate optional extensions, and 100-110 are held out for final testing. This produces 1,200 training and 660 test windows. The non-tied training labels are FCFS 63, SCAN 267, and SSTF 800; 70 additional windows are ties. C-SCAN never uniquely wins.

Two 40-window shift timelines change after window 20: sequential-to-random and random-to-bursty. Measurements are total seek, tie-aware accuracy, selector regret (selected minus oracle cost), confusion matrix, confidence on correct versus wrong predictions, reliability, and 10-bin expected calibration error (ECE) [3]. Python, NumPy, pandas, Matplotlib, and scikit-learn implement the deterministic pipeline [4].

3. Overall and Per-Workload Results

Policy	Total seek	Policy	Total seek
FCFS	902,104	C-SCAN	251,307
Always SSTF	133,276	Oracle	119,296
SCAN	144,737	Learned selector	124,574

Table 1. Total seek on all 660 held-out windows. Always SSTF is the strongest fixed-policy baseline.

The selector saves 8,702 tracks (6.53%) relative to always choosing SSTF. It is 5,278 tracks above the oracle over all windows, or 8.00 tracks per window. On 597 non-tied windows, accuracy is 74.37%; accepting any tied oracle winner gives 75.61% over all 660. Accuracy alone is insufficient: a wrong choice can have nearly the same seek cost as the winner, while an uncommon error can be expensive.

Workload	FCFS	SSTF	SCAN	C-SCAN	Oracle	Selector	Acc.
Sequential	44,549	38,504	46,291	84,450	35,020	39,214	62.7%
Random	729,441	63,558	54,054	86,664	53,398	54,098	70.9%
Bursty	128,114	31,214	44,392	80,193	30,878	31,262	93.2%

Table 2. Per-workload totals, 220 held-out windows per workload.

The per-workload table exposes the weakness honestly. Sequential accuracy is only 62.7%, with mean regret 19.06, compared with 3.18 for random and 1.75 for bursty workloads. Nevertheless, the selector nearly matches the oracle for random and bursty input. The confusion matrix shows most errors involve SSTF versus FCFS or SCAN.

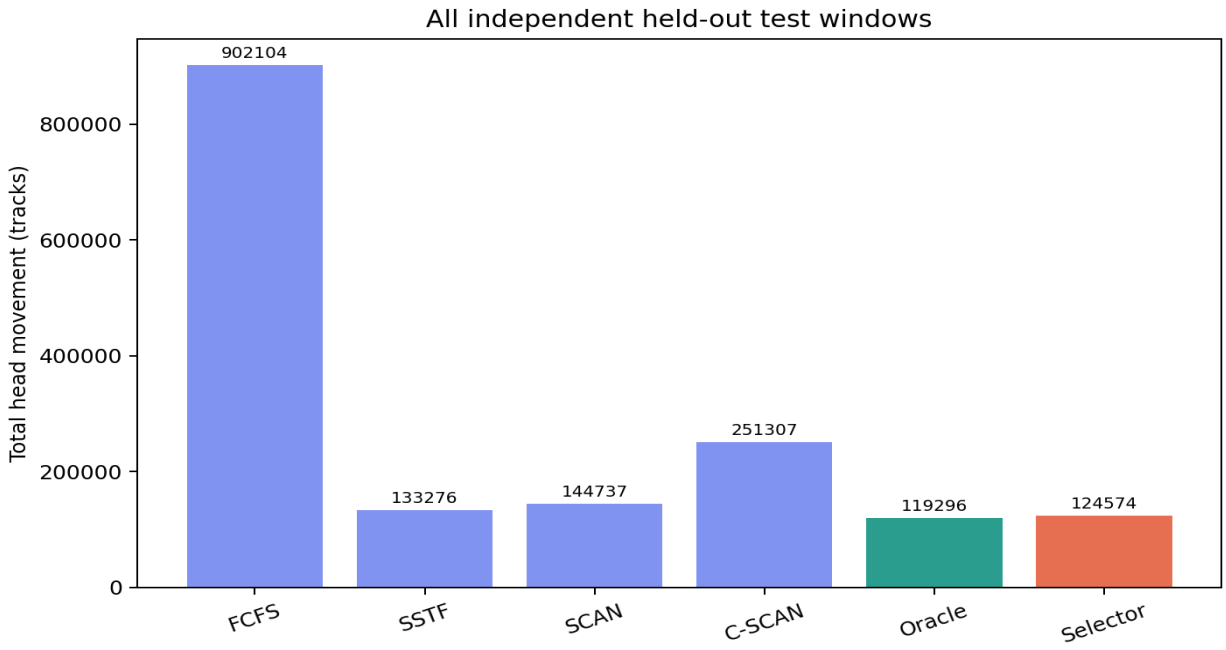


Figure 1. Total head movement. The learned selector beats every single fixed policy.

C-SCAN never wins under this convention because the head starts near the lower end (50), movement begins toward 0, and the complete 0-to-199 wrap is charged. This fixed wrap is expensive for finite windows. This is a consequence of the stated geometry and seek-only objective, not a universal claim: fairness and starvation are outside the measured cost.

4. Confidence, Errors, and Workload Shift

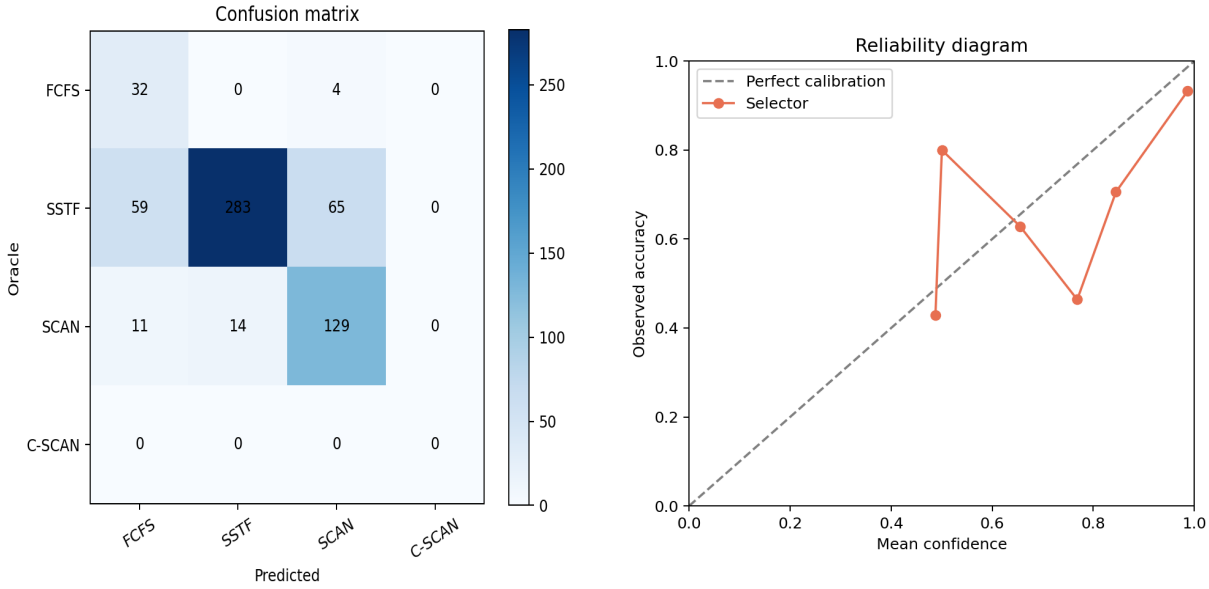


Figure 2. Non-tied confusion matrix (left) and reliability diagram (right).

Confidence is informative but imperfect. The main tree averages 0.877 when correct and 0.778 when wrong. ECE=0.148 and the reliability curve reveal overconfidence in some bins. Thus confidence helps identify risky decisions, but it is not a probability guarantee.



Figure 3. Mean regret and confidence across the two workload shifts; the dashed line marks the change after window 20.

Under the sequential-to-random shift, mean regret falls from 19.06 to 3.18 after the change while confidence falls from 0.861 to 0.809. Under random-to-bursty, regret falls from 4.32 to 1.49 and confidence rises from 0.799 to 0.889. The selector therefore reacts to changed request features immediately at each independent window, although sequential behavior remains the difficult case.

5. Validation-Controlled Extensions

Four additional features target sequential structure: absolute correlation between request index and track, fraction of consecutive gaps at most five, median gap, and maximum gap. A logistic-regression baseline and a 300-tree random forest were trained on seeds 1-20. Selection used only seeds 21-30; the forest won with 78.5% validation accuracy and ECE 0.028, and was then evaluated once on the held-out test set.

Model	Test acc.	Seq. acc.	Mean regret	ECE	Conf. right / wrong
Decision tree (main)	75.61%	62.73%	8.00	0.148	0.877 / 0.778
Logistic baseline	73.33%	57.27%	10.44	0.113	0.853 / 0.738
Random-forest extension	79.09%	70.91%	8.90	0.051	0.827 / 0.615

Table 3. Test results after validation-only extension selection.

The forest improves accuracy, sequential recognition, calibration, and confidence separation, but its total seek is 125,170 and mean regret 8.90, slightly worse than the main tree. Logistic regression is weaker. The tree remains the primary result because it is simpler and better on the measured seek objective; the forest is a calibration-focused extension. This contrast demonstrates that the most accurate classifier need not minimize systems cost.

Window	Tie-aware acc.	Seq. acc.	Mean regret	Best fixed
30	81.52%	82.27%	4.05	SSTF
50	75.61%	62.73%	8.00	SSTF
100	84.39%	89.55%	2.95	SSTF

Table 4. Window-size sensitivity after retraining at each size.

Performance is sensitive to window length, but the main ranking is stable: SSTF remains the best fixed policy at 30, 50, and 100 requests, while the learned selector remains better than that fixed baseline. The 50-request setup is retained because it is the pre-specified main experiment.

6. Limitations and Conclusion

This synthetic simulation measures seek distance only. It omits rotational latency, caching, SSD behavior, concurrent arrivals during service, priorities, starvation, and device timing. Complete windows and an offline oracle simplify a real online problem; the fixed head and SCAN direction limit generalization. Future work should use rolling windows, multiple head positions, cost-sensitive training, and calibration learned on a dedicated calibration set.

Within these limits, the decision-tree selector improves over every fixed scheduler and approaches the oracle while exposing useful confidence information. All code, seeds, raw outputs, figures, tests, and an interactive custom-queue demonstration are included for audit and reproduction.

AI-use disclosure. Generative AI assisted with scaffolding, implementation, documentation, and debugging. All numerical claims were regenerated by deterministic code and checked against committed CSV/JSON outputs. The student remains responsible for understanding and defending the work.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
[2] L. Breiman et al., *Classification and Regression Trees*, Wadsworth, 1984.
[3] C. Guo et al., 'On Calibration of Modern Neural Networks,' ICML, 2017.
[4] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' JMLR 12, 2011.